CORE FITNESS HELPER

Product Blueprint & Implementation Plan

A cross-platform fitness and gym-management platform connecting members, trainers, and gym owners through workouts, exercise references, progress tracking, nutrition, memberships, payments, and trainer bookings.

Platforms	Android and iOS mobile app
Admin experience	Responsive React web dashboard
Suggested mobile stack	React Native + TypeScript
Suggested backend	Firebase services for the initial version
Document status	Initial planning baseline — decisions can be refined before implementation

Product direction: make exercise references, workout plans, and progress tracking the core member experience, while supporting the essential gym operations needed for a pilot gym.

1. Executive Summary

Core Fitness Helper is planned as an all-in-one fitness platform. A single role-aware mobile app serves members and trainers; gym owners and administrators use a web dashboard for management-heavy tasks.

- **Members:** discover exercises, follow workout plans, log sets and reps, track progress, record nutrition, manage memberships, pay fees, and book trainers.
- **Trainers:** manage assigned clients, create workout plans, view relevant progress data, maintain availability, and handle sessions.
- **Gym owners/admins:** manage gyms and branches, members, staff, packages, payments, bookings, attendance, and reports.
- **Platforms:** React Native for Android and iOS, plus a responsive React web dashboard.
- **Initial backend recommendation:** Firebase Authentication, Cloud Firestore, Cloud Storage, Cloud Functions, and Firebase Cloud Messaging.

> **Planning principle**
>
> Deliver a reliable fitness core first, then layer on commercial and administrative workflows. Use a pilot gym to validate usability and operational rules before broad rollout.

2. User Roles and Responsibilities

Role	Core capabilities
Member	Exercise library with images/videos; workout plans; active workout logging; body measurements and progress; diet log; membership details; payments and receipts; trainer booking.
Trainer	Assigned client list; client profiles and member-shared logs; workout plan creation and assignment; schedule and availability; booking management; session notes.
Gym owner / admin	Gym and branch configuration; member and staff management; membership packages; renewals; payment reconciliation; booking calendar; attendance; revenue and activity reports.
Platform administrator (optional operational role)	Support and platform-level configuration across gyms. This role should be tightly restricted and audited, separate from individual gym owners.

Role design

Use gym-scoped roles rather than one global role per user. A person may belong to multiple gyms and may hold different roles in each. Enforce permissions on the backend and database rules, not only in navigation or UI visibility.

3. Screen Inventory

3.1 Member mobile screens

- **Home dashboard:** today's workout, membership status, upcoming trainer session, weekly activity, and quick actions.
- **Workout library:** search/filter by exercise name, muscle group, equipment, and difficulty; show images, video demonstrations, instructions, and safety tips.
- **Workout plans:** weekly schedule, ordered exercises, sets, reps, target weights, rest timer, and workout completion.
- **Progress:** body weight and measurements, workout consistency, personal records, and trend charts.
- **Diet and nutrition:** manual food diary, calorie/macronutrient estimates, meal-plan view, and water intake.
- **Bookings:** trainer profiles, availability, booking confirmation, rescheduling/cancellation rules, and session history.

- **Membership and payments:** active package, start/end dates, renewal, invoices/receipts, payment history, and outstanding amounts.
- **Profile and settings:** goals, preferences, notification settings, privacy, language, and account controls.

Suggested bottom navigation: Home · Workouts · Progress · Bookings · Profile. Place nutrition, membership, and payment details within dashboard shortcuts and relevant sections.

3.2 Trainer mobile screens

Screen	Key functions
Trainer dashboard	Today's sessions, pending requests, assigned clients.
Client list/profile	Search clients; view assigned clients and relevant member-shared progress.
Plan builder	Create workout routines; select exercises, sets, reps, rest periods; assign plans.
Schedule	Set availability; view bookings; handle approved cancellations.
Session notes	Mark attendance/completion and save follow-up notes.
Trainer profile	Specialties, qualifications, pricing where applicable, availability.

3.3 Gym owner/admin web screens

Screen	Key functions
Overview	Active members, upcoming renewals, revenue summaries, attendance.
Members	Registration, membership status, assignments, search and bulk operations.
Membership plans	Duration, pricing, benefits, renewal rules and applicable taxes.
Payments	Payment statuses, receipts, refunds, reconciliation and outstanding balances.
Trainers and staff	Staff profiles, gym-scoped roles, schedules and trainer assignments.
Bookings	Calendar, availability, conflicts, cancellations and attendance.
Exercise library	Manage shared exercise metadata, images and videos.
Reports/settings	Revenue/activity reports, branches, tax settings and gym configuration.

4. Key User Journeys

4.1 Member onboarding and workout completion

- Sign up and verify email or phone.
- Join a gym or accept a gym invitation.
- Set goals, experience level, and equipment preferences.
- Receive a trainer-assigned plan or choose an available plan.
- Open exercise instructions and view the image/video demonstration.
- Record sets, reps, weights and completion; save safely if offline.
- Review progress and receive optional workout reminders.

4.2 Membership and payment

- Choose a package and review duration, price, taxes and terms.

- Create a payment order on the trusted backend and complete payment through the gateway.
- Verify payment using server-side gateway verification and signed webhooks.
- Activate the membership only after verified payment or an authorized admin action.
- Show receipt, membership dates, renewal status and reminders.

4.3 Trainer booking

- Browse trainer profiles and specialties.
- Choose an available time slot.
- Validate and reserve the slot on the server to prevent double-booking.
- Notify the member and trainer; apply clear cancellation/rescheduling rules.
- Track attendance, completion and session notes.

5. Recommended Technology Stack

Layer	Recommendation	Purpose
Mobile	React Native + TypeScript	Shared Android/iOS application and reusable UI.
Navigation	React Navigation	Role-aware navigation flows.
Server state	TanStack Query	Fetching, caching and invalidation of API/server data.
Forms/validation	React Hook Form + Zod	Consistent input validation and typed forms.
Authentication	Firebase Authentication	Identity, login and account recovery.
Database	Cloud Firestore	Gym, exercise, workout, booking and membership metadata.
Media	Cloud Storage	Exercise images, video files and user-selected profile images.
Trusted backend	Cloud Functions	Payment verification, privileged changes, booking validation, notifications and scheduled tasks.
Push	Firebase Cloud Messaging	Booking, workout and renewal notifications.
Admin web	React + TypeScript	Reporting, bulk management and configuration.
Payments	Razorpay or another supported gateway	Payment orders, verification, refunds and reconciliation.
Testing/monitoring	Jest, React Native Testing Library, Detox, Crashlytics	Unit/component/E2E tests and crash diagnostics.

Keep local UI state simple. Add a global client-state library only if concrete shared-state needs arise. For offline workout drafts, use a persistent local store (for example SQLite) and design synchronization/idempotency explicitly.

6. High-Level Architecture

Mobile and web clients should call trusted backend functions for privileged actions. Clients can read or write directly to Firestore only where the security rules explicitly authorize the operation. Payment confirmation, role changes, booking conflict checks, and membership activation must not depend on untrusted client claims.

React Native mobile app (member + trainer)	React web dashboard (owner + admin)

Authenticated client access with strict rules	Authenticated client access with strict rules
Firebase Authentication · Firestore · Cloud Storage	
Cloud Functions / trusted backend	
Payment gateway · FCM · scheduled jobs · audit logging	

This is a conceptual architecture. Exact direct-access patterns should be decided per collection and operation after defining the security rules.

7. Initial Data Model

Recommended top-level entities and example fields. This is a starting contract, not a complete production schema.

Entity	Core fields / notes
User	uid, displayName, email/phone, photoURL, account status.
Gym	gymId, name, branchIds, status, settings.
GymMembership	gymId, userId, role, status, joinedAt; role is scoped to a gym.
Exercise	name, muscleGroups, equipment, difficulty, instructions, imageUrls, videoUrl, visibility.
WorkoutPlan	gymId/ownerId, name, createdBy, ordered exercise prescriptions, visibility.
WorkoutLog	userId, planId, startedAt/completedAt, exercise entries, sets, reps, weight, completion.
BodyMeasurement	userId, measuredAt, metric type, value, unit; private by default.
NutritionLog	userId, date/time, food items, estimated calories/macros, source.
MembershipPlan / Subscription	gymId, package details; member subscription period, status, renewal dates.
TrainerBooking	gymId, memberId, trainerId, start/end, status, cancellation metadata.
Payment	gymId, userId, amount in minor currency units, currency, gateway IDs, status, timestamps.
AuditLog / Notification	Actor, action, target, timestamp; notification delivery state and references.

Firestore structure can use gym subcollections for tenant-scoped data and separate user-owned collections for private workout/nutrition history. Avoid duplicating sensitive fields without a clear access-control and synchronization strategy.

8. Security, Privacy and Reliability

- **Tenant isolation:** a gym user must not read or modify another gym's private data unless explicitly authorized.
- **Server-side authorization:** enforce roles and ownership in Firestore Security Rules and Cloud Functions; do not trust role values supplied by the client.
- **Payment integrity:** verify signatures/webhooks, process events idempotently, retain an auditable payment history and reconcile gateway status.
- **Booking integrity:** validate availability on the server and use transaction/locking strategies to avoid double-booking.
- **Private data:** restrict measurements, nutrition logs and client progress to the member and specifically authorized trainers/admins.

- **Offline sync:** queue workout drafts, resolve duplicate retries safely, and surface sync failures rather than silently dropping entries.
- **Data lifecycle:** define retention, account deletion/export, backups, media access rules and incident response.
- **Notifications:** avoid sensitive health or payment details in lock-screen notification text.
- **Operational readiness:** monitor errors, audit privileged actions, test access rules and prepare a recovery plan.

Nutrition and fitness features should be framed as general tracking and educational support, not diagnosis or individualized medical treatment. Any medical or injury-related recommendations need appropriate professional oversight.

9. Proposed Implementation Roadmap

Indicative timeline for a small, focused team. Scope, design readiness, content preparation, payment onboarding and testing can change these estimates.

Phase	Indicative timing	Deliverables
1. Foundation	Weeks 1–2	Repo structure, design system, Firebase environments, authentication, gym onboarding, role-aware navigation.
2. Fitness core	Weeks 3–5	Exercise library with images/videos, plan builder, active workout logging, history and progress charts.
3. Gym operations	Weeks 6–8	Membership plans, trainer scheduling/bookings, admin web basics, member management and notifications.
4. Commercial features	Weeks 9–11	Gateway integration, server verification, receipts, renewals, nutrition logging, reports and access-control review.
5. Pilot and release	Week 12+	Android/iOS device testing, E2E tests, privacy disclosures, pilot gym rollout, feedback and fixes.

Suggested release priorities

- **Must-have for a usable pilot:** authentication, gym-scoped roles, exercise references, workout plans, workout logs, member/trainer basics, memberships, booking flow and admin essentials.
- **Include where operationally required:** payment gateway, receipts and renewal notifications; payment integration should be tested end-to-end before real money is accepted.
- **Keep basic at first:** nutrition diary with manual entries and estimated totals.
- **Postpone:** AI-generated plans, wearable integrations, live video calls, advanced analytics, barcode scanning, social feed and leaderboards.

10. Suggested React Native Project Structure

```
src/
  app/
    navigation/
    providers/
    config/
  features/
    auth/
    gyms/
    members/
    trainers/
    exercises/
    workouts/
    progress/
    nutrition/
    memberships/
    payments/
    bookings/
  shared/
    components/
    hooks/
    theme/
    utils/
    validation/
  services/
    firebase/
    api/
    notifications/
  types/
```

Keep screens, repositories, business rules and validation separate. Organize by feature so the codebase remains navigable as modules expand.

11. Testing and Release Checklist

Area	Minimum validation
Authentication & roles	Unauthorized roles cannot access protected screens or backend data; membership in one gym does not grant access to another.
Exercise/workouts	Images/videos load; plan ordering is stable; sets and reps persist; repeated offline retries do not create duplicate logs.
Bookings	Overlapping slots are rejected; time zones and daylight/time changes are handled; cancellations follow policy.
Payments	Failed, pending, duplicated and delayed webhooks are handled; only verified payment activates paid access; refunds reconcile.
Memberships	Start/end dates, renewal, expiry, grace period and admin adjustments behave consistently.
Privacy	Member data is private by default; trainer access is assignment-scoped; account export/deletion processes are defined.
App quality	Android/iOS device coverage, slow networks, accessibility, crash reporting, app upgrade and recovery tests.
Operations	Production secrets are protected; rules are tested in a staging environment; backups and monitoring are configured.

12. Decisions to Confirm Before Development

The plan uses the recommendations below as a baseline. Confirm them before detailed implementation and estimates.

Decision	Recommended baseline
Product scope	Complete core gym platform; prioritize fitness core and a single-gym pilot.
Mobile app	One React Native + TypeScript app with role-aware navigation.
Admin experience	React web dashboard for owners/admins.
Backend	Firebase for initial delivery; review Firestore cost, query needs and relational reporting before scaling.
Theme	Dark theme with green accents as the initial visual direction; support a coherent design system.
Payments	Select a gateway after checking business onboarding, supported payment methods, refund flow and applicable policies.
Workout media	Use licensed or original exercise images/videos; define upload moderation and storage rules.
Localization	Start with English-ready architecture; assess Tamil localization for the initial audience.
Gym model	Decide whether launch is single-gym pilot, multi-gym SaaS, or both from day one; tenant boundaries must be designed accordingly.

Recommended next step

Turn this baseline into a screen-by-screen functional specification, followed by the navigation map, finalized database/security rules, UI wireframes, and a prioritized development backlog. Build and validate the fitness core before expanding the operational modules.

Document prepared as an initial planning artifact. Product decisions, timelines, pricing, legal requirements, and gateway availability should be validated before launch.